

AGENTIC ENGINEERING IN GITHUB COPILOT CLI

A Reusable Workflow for Shipping Real Code with AI Agents

Grounded in domain-driven design, clean architecture, and four decades of engineering practice

Synthesized from Matt Pocock's "AI Coding for Real Engineers"

the mattpocock/skills repository (main @ 66898f6, July 2026), and Burke Holland's GitHub Copilot CLI workflow

GitHub Copilot CLI edition · Repository-agnostic · Practitioner's guide · July 2026 edition

Contents

CONTENTS.....	2
PART I · OWN YOUR PROCESS.....	5
THE “CODE IS CHEAP” THESIS IS WRONG.....	5
HOW TO READ THIS GUIDE	5
PART II · HOW THE AGENT ACTUALLY WORKS	6
CONSTRAINT 1 — ATTENTION DEGRADES: THE EFFICIENT ZONE AND THE INEFFICIENT ZONE	6
CONSTRAINT 2 — HALLUCINATION.....	6
CONSTRAINT 3 — YOU CANNOT USE AN LLM AS A DATABASE	6
CONSTRAINT 4 — STATELESSNESS	7
A FIFTH THING TO EXPECT — NON-DETERMINISM	7
THE HARNESS, SESSIONS, AND TURNS	7
SUBAGENTS — DELEGATION THAT PROTECTS CONTEXT	8
CONTEXT HYGIENE — CLEAR, DON’T COMPACT.....	9
STEERING — KEEPING A STATELESS AGENT ON YOUR RAILS	9
<i>Keep the standing brief ruthlessly small.....</i>	9
<i>Progressive disclosure</i>	10
<i>Agent skills — the built-in form of progressive disclosure.....</i>	10
<i>Writing great skills — predictability is the root virtue</i>	10
<i>Automatic memory.....</i>	11
AX — AGENT EXPERIENCE	11
PART III · FOUR FAILURE MODES AND THE ENGINEERING CANON	12
FAILURE 1 — “THE AGENT DIDN’T DO WHAT I WANT”	12
FAILURE 2 — “THE AGENT IS WAY TOO VERBOSE”	12
FAILURE 3 — “THE CODE DOESN’T WORK”	12
FAILURE 4 — “WE BUILT A BALL OF MUD”	13
THE MAP FROM FAILURE TO FIX TO CANON	13
PART IV · THE SEVEN-PHASE WORKFLOW.....	14
PHASE 1 — GRILL: HARDEN THE IDEA.....	14
PHASE 2 — RESEARCH: CACHE THE EXPENSIVE EXPLORES	15
PHASE 3 — PROTOTYPE: MAKE IT CONCRETE, APPLY TASTE.....	16
PHASE 4 — SPEC: WRITE THE DESTINATION	16
PHASE 5 — TICKETS: DON’T PLAN, KANBAN	17
<i>Tracer bullets — the unit of a ticket.....</i>	17
PHASE 6 — IMPLEMENT: HITL FIRST, THEN AFK	18

<i>The red / green loop (refactor moved to review)</i>	18
<i>Pre-commit hooks — a deterministic safety net</i>	18
<i>Browser tools — visual feedback loops</i>	18
<i>Going AFK — autopilot and the Ralph loop</i>	19
<i>Pulling from a backlog</i>	19
<i>Sandboxing</i>	19
<i>Handoff — expand, then contract</i>	20
PHASE 7 — REVIEW AND CONTINUOUS IMPROVEMENT	20
<i>Deep modules — designing for AX</i>	21
THE WAYFINDER MAP — WHEN THE IDEA IS BIGGER THAN A SESSION	22
PART V · THE SKILL LIBRARY	24
HOW THE SKILLS COMPOSE	24
ENGINEERING SKILLS	24
PRODUCTIVITY SKILLS	26
MISC SKILLS	26
SKILLS TO WATCH	27
PART VI · PUTTING IT INTO PRACTICE	28
PHASE 0 — SETUP (ONCE PER MACHINE, ONCE PER REPO)	28
GREENFIELD VS BROWNFIELD	28
FROM HITL TO AFK — EARN THE AUTONOMY	28
THE GRILL / EXECUTE / CLEAR RHYTHM	28
WORKING AS A TEAM	28
MAINTENANCE CADENCE	29
PART VII · SANDCASTLE — THE AFK SOFTWARE FACTORY	30
WHY SANDCASTLE EXISTS	30
THE MENTAL MODEL AND VOCABULARY	30
QUICK START	31
THE RUN() API, IN BRIEF	31
SANDBOX PROVIDERS	31
THE PIPELINE IT UNLOCKS: PLAN → IMPLEMENT → REVIEW → MERGE	32
HOW IT FITS THE WORKFLOW	32
NATIVE AUTOPILOT AND THE COPILOT CLOUD AGENT	32
SIDEBAR: RUNNING SANDCASTLE WITH GITHUB COPILOT CLI	33
GOTCHAS WORTH KNOWING	33
WHEN TO USE IT — AND WHEN NOT	33

APPENDIX A - VOCABULARY	35
APPENDIX B - THE READING LIST.....	38
APPENDIX C - COMMAND QUICK REFERENCE	39
SOURCES	41

Part I · Own Your Process

Since late 2025, coding agents have taken a massive leap in capability — and a wave of heavyweight frameworks (GSD, BMAD, Spec-Kit and the like) has rushed in to “own” the process for you. The premise of this guide is the opposite: you, the engineer, should own your process. What follows is a process you can run today, then iterate on and make your own.

The approach rests on a single conviction: software-engineering fundamentals matter more in the AI age, not less. The heuristics built up over thirty-plus years of industry experience — domain-driven design, tracer bullets, test-driven development, deep modules, designing the system every day — are not legacy baggage to be automated away. They are the rails that make an agent reliable. Throughout this guide we will repeatedly wave actual books at you, because the classics already contain the answers; they simply apply harder to a stateless, forgetful agent than they ever did to a human.

The “code is cheap” thesis is wrong

A popular argument says that because AI churns out code so cheaply, code quality matters less — you can simply regenerate your way out of a bad codebase. This is backwards. Quality matters more now, and the agent is more sensitive to it than a human, for three reasons:

- **It is stateless.** A human survives a bad codebase by accumulating memory of where things live and why. A cleared agent has none of that — it is “a new developer joining your team every twenty minutes with zero institutional knowledge.” A garbage codebase gives it no chance.
- **It copies your codebase by default.** Of the three inputs an agent has — your prompt, your steering files, and the codebase itself — the codebase is the strongest signal. The agent imitates the patterns it already sees, even when your steering explicitly tells it not to.
- **It cannot make the entropy-reducing commits.** Agents are excellent at adding features and fixing bugs, and poor at the rarer, harder redesign commits that pay down complexity. So they are a software-entropy accelerator: more commits, faster, most of them nudging the “kludge meter” upward.

The principle. Garbage in, garbage out. Cheap code just accelerates you toward a maxed-out kludge meter — a codebase so complex that even the AI can no longer change it. Investing in quality is now a force multiplier, not a tax.

How to read this guide

Part II explains how the agent actually works — the constraints that every later decision is designed around. Part III reframes those constraints as four failure modes and ties each to the engineering canon. Part IV walks the seven-phase workflow end to end, and closes with the wayfinder map — the second front door for work too big for one session. Part V is a reference to the skill library; Part VI is the practical setup and run-book, including how the workflow scales to a team. Part VII is the AFK setup and Sandcastle process. Appendices give a vocabulary, a reading list, and a command reference. Read Parts II–III once for the mental model; thereafter Parts IV–VII and the appendices are the working reference.

Part II · How the Agent Actually Works

“Large language models aren’t junior developers. They’re something fundamentally weirder and more constrained than that.”

If you don’t understand the constraints, “you’re going to end up blaming AI when in fact you’re just working against how the models are supposed to work.” Four interlocking constraints drive the entire workflow. Fight them and you get slop; design around them and the agent becomes genuinely powerful.

Constraint 1 — Attention degrades: the efficient zone and the inefficient zone

A model does not merely store tokens; it stores every token’s relationship to every other token. Add one token and the number of relationships grows quadratically — “like adding a team to a football league, where the number of games played massively increases as you add teams.” Four tokens make six relationships; eight make twenty-eight; a hundred make roughly five thousand. Crucially, each token’s attention budget is finite and does not grow as the context grows. So as a session fills, the signal on the relationships that matter gets spread thinner and thinner.

Early in a session the agent is in the efficient zone: sharp, focused, attends to everything, reasons cleanly. As context fills it drifts into the inefficient zone: sloppier, more forgetful, prone to hallucination — and, tellingly, it begins to struggle to recall information sitting right there in its own context window. As a working heuristic, the efficient zone is roughly the first 100,000 tokens or so — on frontier models the inefficient zone commonly begins around 125K–150K, though the exact line is debated, and the slide is a gentle curve rather than a wall. (In the 200k-window era this was “get paranoid around the 40% mark”; some argue 60–70%; everyone agrees the effect is real.) Matt’s own working ceiling corroborates the numbers: keep a day-task session under ~150k. (Matt’s public materials and the AI Coding Dictionary call these the “smart zone” and “dumb zone,” a coinage credited to Dex Horthy.) A useful continue-or-clear heuristic: if a dumber model could finish the task from here, keep going; if not, clear.

Gotcha. Million-token windows (e.g. Opus / Sonnet models) do not move the efficient zone. “All that’s changed is they’ve shipped a lot more inefficient zone.” Think in raw token counts, not percentages, and stay paranoid about context usage. Fable notably has a 200K context, nearly pure efficient zone.

Constraint 2 — Hallucination

Hallucination is not a bug; it is what happens when you ask the model about something outside its fuzzy, compressed knowledge. “The model wants to help so badly that it fills the gaps with plausible-sounding nonsense” — inventing methods, getting an API’s shape subtly wrong. It comes in two flavours: factuality (inventing facts) and faithfulness (drifting from the context it was given).

Constraint 3 — You cannot use an LLM as a database

Picture the model as having “compressed all of human knowledge down to a blurry JPEG — scribbled on the back of a napkin somewhere, and it can only half-remember it.” Recall from training data (parametric knowledge) is, by design, unreliable. But anything in the context window (contextual knowledge) is seen at full fidelity and answered reliably.

The principle. Put the documentation and code examples the task needs directly into context — don't rely on the model's memory. Anything newer than the training cutoff is a fabrication trap unless you load its docs.

Constraint 4 — Statelessness

“LLMs behave kind of like that guy from Memento, where every time he woke up he'd completely forget his entire life.” Every cleared context is a total reset. This is the constraint with the largest downstream consequence: because the agent has no memory, your codebase quality, documentation, and organization become the only persistent institutional knowledge. The real question becomes: is your repository friendly to a brand-new starter who arrives every twenty minutes?

A fifth thing to expect — non-determinism

Agents are next-token-chooser machines selecting by probability, so the same prompt against the same codebase yields different answers. Most responses cluster around something sensible; a few are weird outliers. You can make agents more consistent with feedback loops and tight scopes, but you cannot make them deterministic. “Ride the wave.”

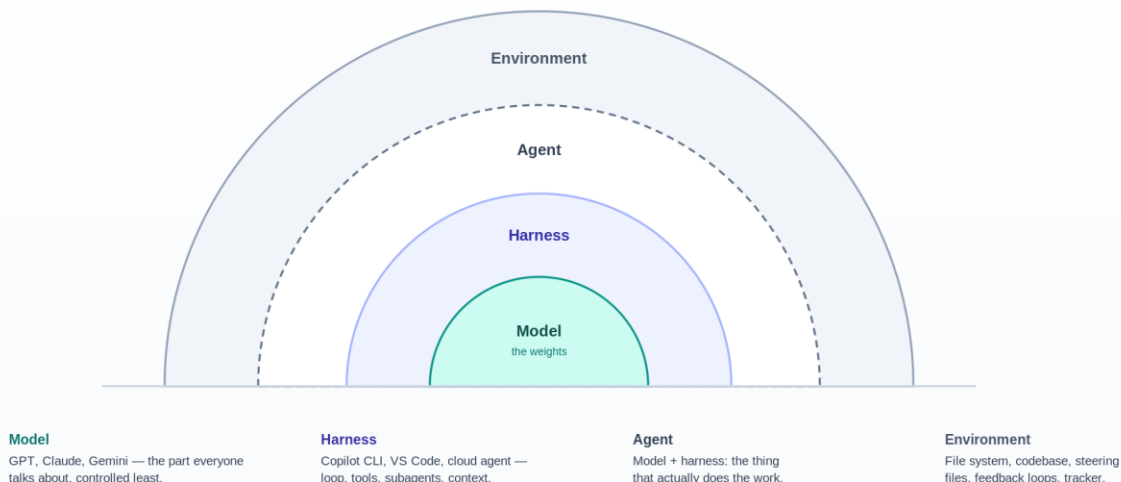
The harness, sessions, and turns

The model itself only does next-token prediction. The harness is everything wrapped around it that turns it into an agent: tools, the system prompt, context-window management, permissions, and hooks. A session is one bounded run that starts empty, accumulates context across turns, and ends when it is cleared, closed, or compacted. A turn is one user message plus everything the agent does before yielding back to you. The workflow is harness-agnostic — any CLI-based agent works — but the examples use GitHub Copilot CLI (generally available since February 2026), typically run inside the VS Code integrated terminal. Copilot CLI's harness adds slash commands, custom agents, and three interactive modes you cycle with Shift+Tab — standard → plan → autopilot (also settable as `--mode=interactive|plan|autopilot`, or `--plan` to start planning) — all of which the workflow below leans on. Model choice is harness-level too: `--model=MODEL` (where `auto` lets Copilot select) or the `COPILOT_MODEL` environment variable, across the Claude, GPT, and Gemini families — the multi-model spread that Phase 7's review stack exploits. Model lists churn fast; pick per task, don't memorize versions.

A useful way to hold the pieces is four concentric rings. The model sits in the middle — the part everyone talks about swapping and the part you control least. Around it sits the harness as defined above. Model plus harness is the agent — the thing that actually does the work. And wrapping the agent is the environment it operates inside: the file system, the codebase, the steering files, the feedback loops, the issue tracker. Chasing model upgrades misses where the leverage is: improving the harness and the environment pays roughly as much as a better model ever will — call it 50/50. You own the environment outright and you choose the harness; the model is the only ring you rent. Everything in this guide — steering files, skills, tickets, feedback loops — is environment and harness engineering.

The rings the agent lives in — model, harness, agent, environment

The model is a small part of the picture — leverage splits roughly 50/50 between the model and everything wrapped around it.



You own the environment and choose the harness — improving them pays as much as a better model ever will.

The whole workflow — steering files, skills, tickets, feedback loops — is environment and harness engineering.

The rings — model, harness, agent, environment: most of the leverage lives in the outer two.

Sidebar: Burke's terminal setup. Burke Holland runs the whole workflow from a full-screen terminal inside VS Code: Command Palette → *Terminal: Create New Terminal in Editor Area* opens the terminal as an editor tab instead of a bottom panel; map it to a shortcut (he uses Ctrl+Alt+Win+T) and summon a full-screen agent instantly. The integrated terminal also has inline chat (Ctrl+I) for recalling a forgotten command, and Ctrl+Q queues your next prompt instead of waiting for the current turn to finish. One editor keeps the agent, the code, Docker, and Remote-SSH in a single place.

Subagents — delegation that protects context

The agent you talk to is the orchestrator (the lead developer). When it needs focused work — usually exploration — it spawns a subagent with a fresh context window. The subagent burns its own tokens in its own efficient zone, then reports a single summary back, so the orchestrator gets the findings without carrying every file read and tool call.

- Subagents can run in parallel, with different prompts and cheaper/faster models — a smaller model on an explore phase is often “fast and really high quality.”
- A subagent runs in its own session and returns one tool result. Subagents can spawn subagents of their own — nesting runs several levels deep — which unlocks model tiering: a big-model orchestrator delegating to mid-size subagents that spawn small explorers, each level roughly an order of magnitude cheaper.

- Word choice triggers depth. “Tell me the tech stack” reads a handful of files; “explore how X works” spawns a subagent that may spend 60k+ tokens and dozens of tool calls and return a summary “almost as good as if we’d written it ourselves.”

Copilot CLI exposes this delegation as first-class machinery. A custom agent —

`.github/agents/NAME.agent.md` in the repo, `~/.copilot/agents/` for personal ones, or org-wide via your organization’s `.github` repository — runs as a subagent with its own context; invoke one with `/agent` or pin it for a whole session with `--agent=AGENT`. `/fleet` fans an objective across several subagents working in parallel — the same exploration trade-off, resolved by the harness itself. And `/subagents` configures per-agent subagent model routing once, instead of specifying it in every prompt — so the model-tiering pattern (a Claude-family orchestrator routing slices to smaller GPT- or Gemini-family workers) becomes standing configuration.

Why it works. There is an exploration trade-off: more tokens on exploring means fewer for implementing, but too little exploring means worse implementation. Subagents resolve it by doing the exploring elsewhere and returning only the conclusion. Because the agent is stateless, exploration is the foundation of every interaction — not a one-time setup.

Context hygiene — clear, don’t compact

When a session fills, you have three moves, and the order of preference matters:

- **Clearing** ends the session and starts fresh with an empty context. This is the preferred default: more smart-zone time, fewer tokens spent summarizing, more predictable behavior, and cleaner separation between tasks. Good workflows are built to start repeatedly from clean context.
- **Compaction** summarizes the conversation so it can continue (autocompact fires automatically near the limit). It preserves what was implemented, files touched, the ask, pending tasks, and any instruction you give just before it runs (e.g. “next goal is QA”). Reach for it only when you are still actively in the conversation — say, you just finished a big build and want focused QA without re-exploring.
- **Handoff** writes the session to a file so a fresh agent can continue (covered in Part IV). The rule of thumb: handoff forks to a fresh session; compaction continues the current one.

Gotcha. Every compaction leaves “sediment.” Repeated compaction layers summaries on summaries and makes output steadily less predictable — autocompact is the worst offender, pulling in context just to shrink it again. Use compaction to rescue hard-won context for one tough follow-up; never build your workflow on it.

Steering — keeping a stateless agent on your rails

Steering is how you keep a forgetful agent aligned with your team’s standards. The naive approach — one giant `AGENTS.md` — collects sediment of its own: dozens of global instructions, most irrelevant to any given request, all loaded on every request and all competing with the actual prompt.

Keep the standing brief ruthlessly small

`AGENTS.md` (the open standard Copilot CLI reads natively) sits right below the system prompt and loads on every single request. Respect the instruction budget: frontier thinking models follow only ~150–200 instructions with reasonable consistency (smaller and non-thinking models, fewer). So the ruthless minimum is: a one-sentence project description (it acts like a role prompt — “This is a React component library for accessible data visualization”), the package manager if it isn’t the default, and any non-standard build/typecheck commands. “That’s honestly it.”

- **Never auto-generate it.** Init scripts flood the file with “useful for most scenarios” cruft, prioritizing comprehensiveness over restraint.
- **Never document file paths.** They are the worst kind of staleness: “authentication lives in src/auth/handlers.ts” becomes a lie the moment someone renames the file, and the agent reads that lie on every request. Describe capabilities and shape, not structure; let the agent generate just-in-time docs during planning.

Progressive disclosure

Borrowed from UI/UX: a bad interface dumps all hundred options into one flat list; a good one reduces the upfront choices and lets you navigate to the rest. Applied to steering, move grouped rules out of the standing brief into separate linked files, leaving only context pointers (“For TypeScript conventions, see docs/typescript.md”). The React Router rules load only when the agent is writing React Router code. Files can nest and can link out to external docs; agents navigate doc hierarchies efficiently.

Agent skills — the built-in form of progressive disclosure

A skill is a folder (a SKILL.md plus optional scripts, docs, and images) that the agent can discover. By default the agent sees only the skill’s name and description; it loads the full instructions only when it actually calls the skill. Skills can themselves link to deeper files, so a lean SKILL.md can sit in front of an entire framework’s documentation. Copilot CLI reads project skills from `.github/skills/`, `.claude/skills/`, or `.agents/skills/` (project-local, shareable via git), and personal skills from `~/copilot/skills/` or `~/agents/skills/`; manage them in-CLI with `/skills add | remove`.

Skills split on who can invoke them:

Type	Who invokes	Purpose	Frontmatter
User-invoked	the human (you type it)	Change behavior: write a spec, run TDD, plan a refactor. These orchestrate.	<code>disable-model-invocation: true</code>
Model-invoked	you or the agent	Hold reusable discipline: validation rules, framework guidance, a rare-error fix.	<code>user-invocable: false</code> to hide from the menu

A user-invoked skill may invoke model-invoked ones, but never another user-invoked skill. This is the seam the whole library is organized around.

Writing great skills — predictability is the root virtue

A skill exists to wrangle determinism out of a stochastic system. The root virtue is predictability — the agent taking the same process every run, not producing the same output — and every authoring decision below serves it.

The invocation choice is an economics question. A model-invoked skill keeps a description the agent sees on every request: it buys autonomy with context load, and every context pointer carries unpredictability — “the model may just choose not to follow it.” A user-invoked skill costs nothing in context but spends cognitive load: you are the index that must remember it exists. Prefer user-invoked for orchestration; when user-invoked skills multiply past what you can remember, the cure is a router skill — which is exactly what `/ask-matt` is.

Structure: steps and reference. A skill’s content decomposes into steps (ordered actions, each ending on a checkable completion criterion) and reference (rules and vocabulary). Reference every run needs stays inline in SKILL.md; reference only some branches reach moves behind a context pointer into a

sibling file. Co-locate a concept's definition, rules, and caveats under one heading — single source of truth.

Steer with leading words. A short, loaded phrase — “tracer bullet,” “vertical slice” — packs a dense concept the model already knows into two words. The agent repeats leading words back in its reasoning, which doubles as your verification that the skill is actually steering.

Hide the future; prune the no-ops. Agents under-invest in a step when they can see the goal past it — the real diagnosis of plan mode's weakness, and the reason the grill and the spec are separate skills: the agent only sees one step at a time. And prune relentlessly with the deletion test: if removing a sentence wouldn't change the agent's behavior, it's a no-op — kill it. `/writing-great-skills` holds the full vocabulary.

Automatic memory

Copilot CLI's experimental `/memory` layers your steering the same way: user memory (your personal brief), project memory (the checked-in `AGENTS.md`), and auto-generated memory the agent writes to steer itself over time, injected alongside `AGENTS.md`. It is powerful but new, and the worst case is that it drifts out of date with the repo.

Gotcha. Memory and steering files go stale, and stale markdown actively poisons context because the agent trusts it on every request. Audit your auto-generated memory, `research.md`, ADRs, and your glossary every couple of weeks and prune anything that no longer matches the repo. Or better yet, disable it.

AX — agent experience

Underneath every later technique is one idea: agent experience (AX) — how well the environment is set up for an agent to do good work. Where DX (developer experience) optimizes for humans, AX optimizes for a stateless collaborator: deterministic checks it can run, an architecture it can navigate, a domain language it can reuse, and free context it isn't forced to waste. Organizing a codebase for AX is, in Matt's words, “probably one of the highest-impact things you can do.”

Part III · Four Failure Modes and the Engineering Canon

Everything in the workflow exists to defeat one of four recurring failure modes. Each failure has a fix, and each fix is lifted straight from a well-known engineering text. This is the heart of the “real engineers” framing: the classics already solved these problems for human teams; agents just make the lessons non-negotiable.

Failure 1 — “The agent didn’t do what I want”

“No-one knows exactly what they want.” — David Thomas & Andrew Hunt, *The Pragmatic Programmer*

The oldest failure in software is misalignment: you assumed the developer understood you, then saw what they built. With an agent the communication gap is identical. Plan mode does not fix it — it truncates the interview, asking one or two questions before dumping a wall of text that’s easy to skim and approve while still misaligned. Frederick Brooks named the missing thing in *The Design of Design*:

“A design concept isn’t an asset or a plan. It’s the ephemeral understanding shared between all participants.” — Frederick P. Brooks, *The Design of Design*

The fix is grilling: the agent interviews you Socratically, one decision at a time, walking down every branch of the decision tree before any code is written, and recommending an answer for each question. Skills: `/grill-me` (general) and `/grill-with-docs` (against your domain model).

Failure 2 — “The agent is way too verbose”

“With a ubiquitous language, conversations among developers and expressions of the code are all derived from the same domain model.” — Eric Evans, *Domain-Driven Design*

Agents are usually dropped into a project and left to infer the jargon, so they use twenty words where one will do. The fix is a shared, ubiquitous language: a `CONTEXT.md` glossary at the repo root that defines the domain’s terms. Compare “there’s a problem when a lesson inside a section of a course is given a spot in the file system” with “there’s a problem with the materialization cascade.” The concision compounds session after session, and the benefits run deeper than brevity: variables, functions, and files get named consistently; the codebase becomes easier to navigate; and the agent spends fewer tokens thinking because it has a tighter language to think in. Skills: `/grill-with-docs` and `/domain-modeling`; hard, surprising, irreversible decisions are captured as ADRs.

Failure 3 — “The code doesn’t work”

“Always take small, deliberate steps. The rate of feedback is your speed limit. Never take on a task that’s too big.” — David Thomas & Andrew Hunt, *The Pragmatic Programmer*

Even when you are aligned on what to build, the agent can still produce code that doesn’t run — because it is flying blind, with no signal on how its code actually behaves. The fix is the usual tranche of feedback loops: static types, browser access, and automated tests, with a red-green loop at the center. Writing a failing test first — and running it to watch it fail — gives the agent a concrete target it can iterate against, and is “the single most consistent way to improve agent output.” Skills: `/tdd` and `/diagnosing-bugs`; pre-commit hooks make the loops deterministic.

Failure 4 — “We built a ball of mud”

“Invest in the design of the system every day.” — Kent Beck, *Extreme Programming Explained*

“The best modules are deep. They allow a lot of functionality to be accessed through a simple interface.” — John Ousterhout, *A Philosophy of Software Design*

Because agents radically speed up coding, they also accelerate software entropy — codebases get more complex at an unprecedented rate. To an agent, a fragmented codebase of dozens of tiny modules with scattered exports is near-unreadable: it must re-trace the import/export graph from scratch every session and can’t see at a glance where functionality lives, and the fragmentation forces brittle, tightly-coupled tests that break on every change, destroying the very feedback loops the agent depends on. The fix is to care about design every day and to build deep modules: a thin, stable interface hiding a large implementation. Skills: `/codebase-design` (the vocabulary) and `/improve-codebase-architecture` (the every-few-days upkeep). More on deep modules in Part IV, Phase 7.

The map from failure to fix to canon

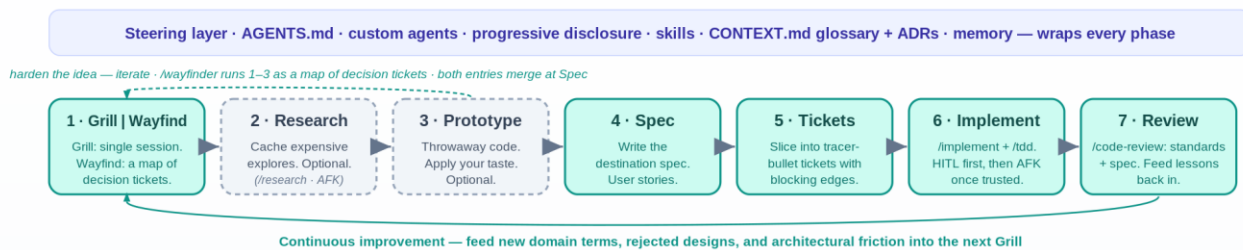
Failure mode	The fix	Skills	Canon
Didn’t do what I want	Grilling — build the shared design concept first	grill-me, grill-with-docs	Brooks, <i>The Design of Design</i> ; <i>Pragmatic Programmer</i>
Too verbose	Ubiquitous language — a CONTEXT.md glossary + ADRs	grill-with-docs, domain-modeling	Evans, <i>Domain-Driven Design</i>
Code doesn’t work	Feedback loops — types, tests, TDD, hooks	tdd, diagnosing-bugs, setup-pre-commit	Beck, <i>XP</i> ; <i>Pragmatic Programmer</i>
Ball of mud	Deep modules — design the system every day	codebase-design, improve-codebase-architecture	Ousterhout, <i>A Philosophy of Software Design</i> ; Feathers (seams)

Part IV · The Seven-Phase Workflow

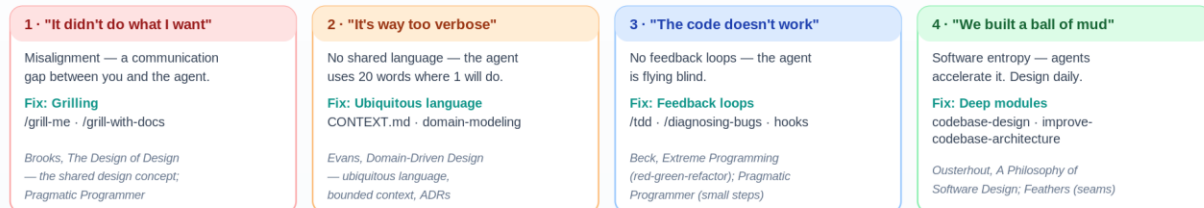
The whole system converges on one through-line: Grill → Research → Prototype → Spec → Tickets → Implement → Review. Phases 1–3 iterate to harden an idea before you commit to a spec; Phases 4–5 turn it into a spec and a backlog of tickets; Phase 6 ships; Phase 7 reviews and feeds lessons back into the next Grill. The steering layer from Part II wraps everything. Phase 1 has two entries — grill the idea that fits one session, or wayfind the one that doesn't: `/wayfinder` runs Phases 1–3 as a map of decision tickets, and both entries merge at the spec. Research and Prototype are optional; the rest are the spine. “AI coding gives you permission to build bigger” — and this is how you exercise that permission without drowning.

Agentic Engineering with GitHub Copilot — The Reusable Workflow

Own your process: real engineering skills, encoded so the agent rides the rails you lay.



Why the workflow exists — the four failure modes & the engineering canon



Human imposes taste · Agent does the grunt work

HITL (human-in-the-loop): grilling, research, prototyping, QA, architecture decisions — where judgment and taste live.
AFK (away-from-keyboard): autopilot — the Ralph-style loop — pulls ready-for-agent issues from the backlog and ships them in a sandbox.

Core constraints the whole workflow is built around

Efficient zone (~first 100k tokens) vs inefficient zone (from ~125–150k) · statelessness ("a new developer every 20 minutes") · hallucination · knowledge cutoff
Clear > compact · subagents protect context · tracer bullets & TDD apply harder to agents, not less

"AI coding gives you permission to build bigger."

Synthesized from Matt Pocock's "Skills for Real Engineers" / "AI Coding for Real Engineers" — adapted for GitHub Copilot CLI · facts re-validated 2026-07-14

The seven-phase workflow, the four failure modes, and the constraints they answer.

Phase 1 — Grill: harden the idea

Goal: reach a precise, shared understanding before any code — the GRILL step of the grill / execute / clear loop. Run `/grill-with-docs` for code work (it also builds your domain model) or `/grill-me` for non-code planning — or `/wayfinder` when the idea is bigger than one session (see the end of this Part): it has become Matt's own default entry for anything non-trivial, and it degrades to a plain grilling session when it senses no map is needed. In Copilot CLI you enter plan mode with Shift+Tab before describing the work, then run the grilling skill so the agent interrogates you one question at a time.

Don't let the agent jump to implementation, and don't trust plan mode's habit of dumping a plan before you actually understand each other. Instead, run a grilling conversation. The skill itself is famously short

— essentially: “Interview me relentlessly about every aspect of this until we reach a shared understanding. Walk down each branch of the decision tree, resolving dependencies between decisions one by one. For each question, provide your recommended answer. Ask one at a time, and if a question can be answered by exploring the codebase, explore instead.” The grilling primitive was reworded in July 2026 to generalize beyond code: it now grills any plan, decision, or idea.

- One question at a time keeps your focus moving decision to decision; a single big plan document invites you to get lazy and skim.
- A recommended answer per question lets you move fast — the most common reply is just “correct.”
- It’s a dialogue, not an interrogation. Drive it. The highest-value move is to ask “what’s reversible — what would be hard to roll back from?” which often flips a recommendation toward the more reversible option.
- Let it explore to answer its own questions rather than pre-answering everything.
- Facts are looked up; decisions are yours. If a fact can be found by exploring the codebase, the agent should look it up rather than ask; every decision, though, gets put to you explicitly. And say “I don’t care” when you truly don’t — it delegates that decision cleanly and keeps the grill moving. The facts/decisions split is also what stops a session from “self-grilling” — exploring the codebase and answering its own questions — and the session ends behind a confirmation gate: nothing gets enacted until you confirm shared understanding.

Why it works. The decision tree comes from Brooks’s *The Design of Design* (his name for it: the design tree): walk every branch before committing to code. Real sessions run roughly 16 questions for a small feature and 30–50 for a complex one. A fully-grilled context window is “pure gold” — if budget remains, implement in that same window; if not, write a spec as the handoff artifact. Don’t clear a grilled context just to write a spec.

Phase 2 — Research: cache the expensive explores

Goal: resolve unknowns once so later runs don’t repay for them. Run `/research` to delegate it: a background agent investigates the question against primary sources — official docs, source code, specs, never a secondary write-up of them — and leaves a cited markdown file in the repo while you keep working. Research feeds the thinking; it doesn’t replace it: the file is something to take into the grill. In Copilot CLI the skill runs naturally as a custom agent, and several open questions can burn down in parallel via `/fleet`; note also the CLI’s new experimental native `/research` command, which does the same job harness-side.

Some work has long explore phases (integrating an external service whose docs aren’t local). Inside an AFK loop spanning many context windows, every run repeats that long explore; over ten runs it gets very expensive. Before the loop, gather what the build needs — external docs, an approach decision (SSE vs WebSockets) — and cache it into a local asset, usually `research.md`. Every later run reads the file instead of re-exploring, keeping runs in the efficient zone with less token spend.

The principle. Treat research as a human-in-the-loop task — your taste decides which library or approach goes into `research.md`, and that judgment shapes everything downstream. Give the file a tightly-managed lifecycle; stale research hurts as much as any stale steering file.

Phase 3 — Prototype: make it concrete, apply taste

Goal: when conversation is too low-fidelity, build a quick, rough artifact to react to — and impose your taste before autonomous implementation. Run `/prototype`, which routes to one of two branches.

- **UI variations** — several radically different designs on one throwaway route, toggled from a floating bar, so you can walk the decision tree by eye (“I like A but take a bit from C”).
- **Logic/state** — a tiny interactive terminal app that pushes a complex or over-time data model through its hard-to-reason cases.

Prototypes work best integrated into the real code they are prototyping for — next to the module or page in question, clearly named as a prototype — because reacting to the real thing beats reacting to a facsimile. Prototype is throwaway by design: one command to run, no persistence, skip the polish — but when it’s done, capture, don’t dispose: fold the validated decision into the real code, then commit the prototype itself to a throwaway branch off main as a primary source, with a context pointer from the implementation issue and the verdict recorded alongside the question it settled. The main branch keeps only the validated decision. It pairs naturally with research — prototype each researched option, QA them, feed the winner into the spec.

For UI work, pair the prototype with a design skill — the community front-end-design skill from `skills.sh`, or your own design language layered on top — and ask for many variations in a single HTML file (Burke Holland generates twenty at once, driving a Claude-family model in the demo) so you can compare them side by side. Preview them in VS Code’s integrated browser with the Live Preview extension, then let the one you pick feed straight into the prompt — and, per the doctrine above, park the mock file on a throwaway branch rather than deleting it: it is a primary source for the implementation ticket.

Gotcha. Prototyping is not for bug fixing (you already know the desired behavior) or for extending existing features (don’t prototype another modal when you have a hundred). Reach for it when you’re redesigning a whole system or facing a genuine “how should this feel?” question that grilling can’t resolve.

Phase 4 — Spec: write the destination

Goal: synthesize everything learned into a reviewable spec that scales a build beyond a single context window. Run `/to-spec`, which turns the current conversation into a spec — the document you may know as a PRD; “spec” is now the single through-line term, since what it holds is broader than product requirements (the skill was `/to-prd` in earlier editions). It does not re-interview you.

The spec has a Problem statement, a Solution, an extensive list of User Stories (desired behavior in plain language, from Agile), Implementation Decisions, Testing Decisions, and an explicit Out of Scope. The user stories are the most important part — they pin down behavior. This version also sketches the major modules, hunts for deep modules (a small interface hiding a large implementation), and asks which modules you want tested and at what boundary. It sketches the seams at which the feature will be tested — preferring existing seams over new ones, placed as high as possible: the fewer seams across the codebase the better, and the ideal number is one. It checks those seams with you, then publishes straight to ready-for-agent — tickets born from a spec don’t need triage.

- Respect ADRs and use the domain glossary — the spec should read in your project’s language.
- No file paths or code snippets in a spec — they go stale fast (the one exception is a decision-encoding snippet lifted from a prototype).
- Commit the spec and skim the user stories for clarity to an outsider before slicing it up. The spec is a summarization artifact — if the grill did its job, alignment already happened, and you rarely

need to study it; polish belongs in QA. When the build ships, close or delete it: a stale spec is do not the agent will trust.

Why it works. A big task bursts into the inefficient zone or doesn't fit at all. The decades-old answer is to break it down so all the work happens in the efficient zone — and that needs a written destination. The spec is where you impose taste and where the build scales beyond one window.

Phase 5 — Tickets: don't plan, Kanban

Goal: turn the destination into a journey the agent can grab piece by piece — a dependency graph, not a rigid multi-phase plan. Run `/to-tickets` to slice, and `/triage` to keep the backlog clean. One bridge for this guide: a ticket is a GitHub Issue, and the wayfinder's decision tickets (end of this Part) are sub-issues of a parent map issue. Triage also covers external PRs — a PR is an issue with attached code: same roles, same states, same machine.

Humans don't rigidly plan everything in one document; once we know the destination we make a Kanban board of issues with blocking relationships. A board is more flexible (add an issue by describing a blocking relationship — no plan rewrite), parallelizable (independent issues run at once), and infinitely extensible (QA feedback becomes new issues that grow the graph organically).

Tracer bullets — the unit of a ticket

An agent's sycophancy makes it want to deliver a complete solution in one leap, building horizontal layers in isolation — all endpoints, then all auth, then all logging — before discovering the database connection string was wrong. The Pragmatic Programmer calls this outrunning your headlights. A tracer bullet is the opposite: a small end-to-end vertical slice that touches every layer at once (schema → API → UI → tests). Build a tiny slice, test it immediately, get feedback, then expand.

"Two well-chosen words can completely change the way AI prioritises and plans. The agent already knows the concept — just say 'use tracer bullets.'"

`/to-tickets` locates the spec (or works from the conversation), explores the codebase, drafts vertical-slice tickets — each declaring its blocking edges — and marks each HITL (needs human judgment) or AFK (can be merged unattended, preferred), always adds a final HITL QA issue blocked by all the others, then quizzes you on granularity, dependencies, and the HITL/AFK split before publishing in dependency order. The same artifact works on two tracker shapes: on a local tracker the edges are text in an ordered `tickets.md` you work top-to-bottom by hand; on a real tracker — GitHub Issues here — they become native blocking links, so any ticket whose blockers are done is on the frontier and multiple agents can work in parallel. Prefer many thin AFK slices over a few thick HITL ones — but do prefer fat vertical slices over thin horizontal ones, which delay feedback on real user value. Everything `/to-tickets` creates is an implementation ticket — the counterpart of the wayfinder's decision tickets, and a distinction worth keeping sharp: a decision ticket can only close by making its decision; an implementation ticket closes when that decision is reified in code.

Two refinements. First, look for chances to prefactor — "make the change easy, then make the easy change." Second, a wide refactor — one mechanical change (rename a column, retype a shared symbol) whose blast radius fans across the whole codebase — is the one exception to vertical slicing: no slice can land green. Sequence it as expand–contract: expand (add the new form beside the old so nothing breaks), migrate call sites in batches sized by blast radius — each batch its own issue, CI staying green because the old form survives — then contract (delete the old form once no caller

remains, blocked by every migrate batch). If even the batches can't stay green alone, keep the sequence but share an integration branch, and promise green only at a final integrate-and-verify issue.

Gotcha. Keep the AFK agent off human-in-the-loop work. A reliable prompt guard: “Work on the AFK issues only, not the human-in-the-loop ones. If all AFK tasks are complete, output `<promise>NO MORE TASKS</promise>`.” Enforce it harder with labels.

Phase 6 — Implement: HITL first, then AFK

Goal: build the slices with a tight feedback loop — watched at first, autonomous once trusted. Kick each issue off with `/implement`, passing it the spec and the single ticket: it drives `/tdd` internally at the pre-agreed seams — one red-green slice at a time — runs typechecks and single test files as it goes and the full suite once at the end, then closes out with `/code-review` before committing; for hard bugs reach for `/diagnosing-bugs`.

The red / green loop (refactor moved to review)

- **Red:** write a failing test first and run it to confirm it fails. This proves the test is wired to real behavior — and, for a bug, proves the bug exists.
- **Green:** write the minimum implementation to pass. The failing test gives the agent a concrete target it can run, inspect, log against, and use as feedback.
- **Refactor:** now deliberately out of the implementation loop — it happens at code review, where the smells are named and fixed in a fresh session, so implementation stays focused. When you do refactor, the old rule holds: keep the suite green, never refactor while red.

Combine red/green with tracer bullets: one test for one vertical slice, make it pass, then the next — never a wide horizontal layer of shallow tests. Test behavior through public interfaces, not implementation details, so tests survive refactors and read like specifications. Tests live at seams — the public boundaries agreed in the spec — never against internals; no test gets written at an unconfirmed seam. Watch for the rhythm (fail before implementation, pass after); it's hard to fake when the test is meaningful.

Pre-commit hooks — a deterministic safety net

`/setup-pre-commit` wires Husky to run typecheck, tests, and lint-staged (Prettier) on every commit; if any step fails the commit is blocked and the agent gets the error back to act on.

The counterintuitive insight. Pre-commit hooks are famously annoying for humans — three minutes of waiting per commit — but perfect for agents. “Claude doesn't care how long it takes. It doesn't get frustrated. The friction that's painful for a human developer is actually perfect for an agent.”

Browser tools — visual feedback loops

When the agent can see its own rendered output, the feedback loop tightens further. This is the visual equivalent of a failing test: a concrete target the agent can run, look at, and correct. Two routes exist, and it pays to know which is which. In VS Code, Copilot's built-in browser tools (generally available since July 1, 2026) let the agent open the page, inspect what it built, and iterate against the real DOM — point it at the running app and describe what's wrong (“the months aren't evenly spaced; there's room for another row of years”). Burke flags the honest limitation: clicking an element in that browser feeds the VS Code sidebar chat, not the CLI — a hole in the CLI workflow he works around by prompting, and the subject of an open feature request. In the CLI itself, browser access goes through

MCP — wire up a browser server such as Playwright MCP and the same iterate-against-the-DOM loop runs from the terminal.

Going AFK — autopilot and the Ralph loop

“Do phase N” is really a for-loop waiting to be automated. Around December 2025 the models became good enough to send off on well-defined tasks, and Geoffrey Huntley’s “Ralph Wiggum” technique — a simple loop running the same prompt over and over to grind through a tranche of work — is the pattern. The key detail: each iteration gets a fresh context window (a bash loop, not one long-lived session held open by a stop hook), so every run stays in the efficient zone. AFK agents let you delegate huge amounts of work while you plan or QA other runs in parallel.

In Copilot CLI the loop is native. Switch to autopilot (Shift+Tab cycles standard → plan → autopilot) and it works through each step until it judges the goal complete, or drive it headless: `copilot -p "<prompt>" --autopilot --allow-all-tools --max-autopilot-continues 10`. Programmatic runs must grant tool access explicitly (`--allow-all-tools`, or the `COPILOT_ALLOW_ALL` environment variable); `--max-autopilot-continues N` caps the iterations — the default is unlimited, so always cap unattended runs; `--no-ask-user` stops the agent pausing on questions; `-s/--silent` emits only the final message for scripting; and `-share/--share-gist` writes the session log somewhere a human can review it later. /fleet fans independent slices across parallel subagents, and for a Ralph loop proper you wrap the headless command in a bash loop so each iteration starts from clean context. (Experimental prompt scheduling — /after and /every — can queue or cron follow-up prompts harness-side; experimental hooks and plugins extend the harness the way Part II’s rings predict.)

Pulling from a backlog

An agent you must point at a plan each time isn’t autonomous. Instead it pulls tasks from a queue — your issues — and decides what to do next, prioritizing in this order: (1) critical bug fixes, (2) development infrastructure (tests, types, dev scripts), (3) tracer bullets (new features), (4) polish and quick wins, (5) refactors. After each task it commits with a detailed message (key decisions, files changed, notes for the next run), closes or comments the issue, and moves on — one task per run. Filter available work with a ready-for-agent label. The pattern holds up to roughly 20–30 open tasks.

Why it works. This is a virtuous loop: the agent produces code, a human reviews it, new issues get created, and the agent picks up the next task — productivity that compounds while you’re elsewhere. One real project (the Sandcastle runner) was built this way across 403 closed issues.

Sandboxing

An AFK agent running without permission prompts has a lot of freedom — it could delete directories or, if prompt-injected, exfiltrate code. A sandbox confines it to the files you share; it can still explore, run bash, and use git, but can’t harm the rest of the system. OS-level sandboxes (Docker, Podman, E2B) are much harder to escape than rules alone — agents can break out of software-only sandboxes if they try. A sandbox-agnostic runner lets you start with no sandbox to focus on the prompt, then add isolation without changing agent code.

In Copilot CLI you keep this safe by suppressing permission prompts only inside the box: `/yolo` (or `--yolo`, now with `on|off|show`) is shorthand for `--allow-all`, which expands to `--allow-all-tools --allow-all-paths --allow-all-urls`. Run the agent in a dev container so the worst case is a destroyed container, and add `--deny-tool` rules as a deterministic guardrail — deny always beats allow, and the `Kind(argument)` syntax scopes precisely, e.g. `--deny-tool='shell(git push)'`.

Handoff — expand, then contract

Mid-session you'll spot a side task — a broken test, a file to update — that deserves its own context window. Clearing would lose the main task; doing it inline may exhaust the budget. `/handoff` writes a temporary markdown doc (in the OS temp dir, not your repo) so a fresh agent can continue: it suggests which skills the next session should use, references existing artifacts instead of duplicating them, and redacts secrets. This enables “expand then contract” flows — hand off from a long grill into a focused sub-session (grill one part, prototype, burn context), then hand a condensed doc back. Reach for it whenever a session drifts into the inefficient zone.

Phase 7 — Review and continuous improvement

Goal: check the work against both the spec and your standards, then feed lessons back in. In this guide the review phase is layered: `/code-review` is the mandatory gate on every ticket; `/rubber-duck` is Copilot's cross-model escalation for high-stakes or design-heavy changes; and Burke's multi-model refinement loop is the hardening pass you run on a finished feature.

Layer 1 — `/code-review`, the gate on every ticket. Review along two axes: Standards (does the code follow this repo's documented conventions?) and Spec (does it faithfully implement the originating spec or ticket?). `/code-review` does exactly this: it pins the diff against a fixed point (a branch, commit, tag, or merge-base), finds the originating spec and the repo's standards sources, and runs both axes as parallel sub-agents so neither pollutes the other. On top of whatever the repo documents, the Standards axis carries a fixed baseline of Fowler code smells (Refactoring, ch. 3) — always judgement calls, never hard violations, and a documented repo standard overrides the baseline. The two reports are never merged or reranked: a change can pass one axis and fail the other, and reporting them separately stops one from masking the other. This is also where refactoring now lives: the implementation does the red-green; the automated review does the refactor, naming the smells and fixing them in a fresh session. And this phase is where the harness earns its keep — with code review, the harness matters more than the model: the pinned diff, the located sources, and the two parallel axes do more for review quality than a smarter model would. The cheapest bug-catcher of all is the second pass itself — reading the finished work against the spec catches what the implementation session cannot see. (Copilot CLI has also grown an experimental native `/review` command — a harness-side complement, not a replacement for the two-axis skill.)

Keep the vocabulary straight: an automated check is deterministic (tests, types, lint, build, pre-commit — pass/fail); an automated review is an agent judging another agent's work (non-deterministic); a human review means reading the diff — reading the summary doesn't count; and accepting code with no review at all is vibe coding.

Where do coding standards live? Push them into the reviewer, not the implementer. The implementing agent's context is already strained — every standard loaded there competes with the actual task — while the reviewer reads them fresh. Run review in a cleared session so it happens in the efficient zone. Mine the standards from your own PR review comments — the things you actually flag — never by asking an agent to generate them from the codebase. And hold one line: humans should only touch AI-reviewed code.

Layer 2 — `/rubber-duck`, the cross-model escalation. Copilot CLI is multi-model, and `/rubber-duck` exploits that: it hands the current plan, implementation, or tests to a different model family than the one driving your session, so the critic doesn't share the orchestrator's blind spots — in Burke's demo, a Claude-family (Opus) planning session critiqued by a GPT-family model (“GPT-5.5”). Different families have different training data and different holes; the hybrid review costs a little more and is far stronger. Reach for it when the change is high-stakes or design-heavy — an architecture decision, a security-sensitive diff, a plan you're about to hand to autopilot — on top of, not instead of, the `/code-review` gate.

Layer 3 — the multi-model refinement loop. To harden a finished feature, run Burke’s review loop on autopilot: ask the agent to review the project for completeness, correctness, security, and improvements; get a contrasting model family’s input on the findings; list them by priority; fix them all; and repeat until both models agree the only items left have diminishing return. That phrase — “diminishing return” — is the stop condition: models always find something when asked to review, so without it the loop never ends. In the demo this ran three rounds and surfaced keyboard accessibility, two-digit year handling, several bugs, year-bound clamping, and HTML escaping — issues that would otherwise have shown up in production. It burns two top-tier models, so spend it on features that warrant it.

Close the loop by mining your own sessions. `/chronicle` (no longer experimental-gated) reviews your session history and suggests how to work better — its subcommands are `standup` | `tips` | `improve` | `search` | `reindex`: `/chronicle tips` surfaces patterns like a prompt you’ve sent a dozen times that belongs in an instruction file, `/chronicle improve` flags where the agent misread your intent, `/chronicle standup` summarizes recent work, and `/chronicle search` (added in v1.0.49) queries the indexed history directly. Bake the suggestions back into your `AGENTS.md` and project skills, and feed new domain terms, rejected designs, and architectural friction into the next Grill — that compounding loop is the whole point.

Deep modules — designing for AX

A deep module (Ousterhout) is a thin interface over a large hidden implementation. Hide as much as possible behind a simple, rarely-changing interface, and the payoff is large: callers depend only on the interface; you can test huge units at the boundary; tests survive internal refactors; and the agent reads one interface to know what the module does and where to change. The skill’s precise vocabulary is worth adopting — Module (anything with an interface and an implementation), Interface (everything a caller must know), Seam (where behavior can change without editing in place — Michael Feathers’ term), Adapter, Depth (how much leverage the interface gives), and Locality — because “vague architecture words create vague refactors.”

`/improve-codebase-architecture` scopes its scan to where change is landing — YAGNI, not a whole-repo audit — and surfaces clusters of shallow modules, spawns parallel sub-agents to design competing interfaces (this is Ousterhout’s “design it twice”), presents the candidates as a visual HTML report, then grills you on the one you pick and records the decision as an ADR / glossary update. Run it every couple of days in a fast-moving repo (weekly at minimum), after a surge of development — and consider a pass during planning, where a deepening opportunity can reshape the tickets. Treat your deep modules as grey boxes: you can look inside, but you trust the implementation by default and review the tests more than the internals — you own the interface, the agent owns the implementation, and the tests keep it honest.

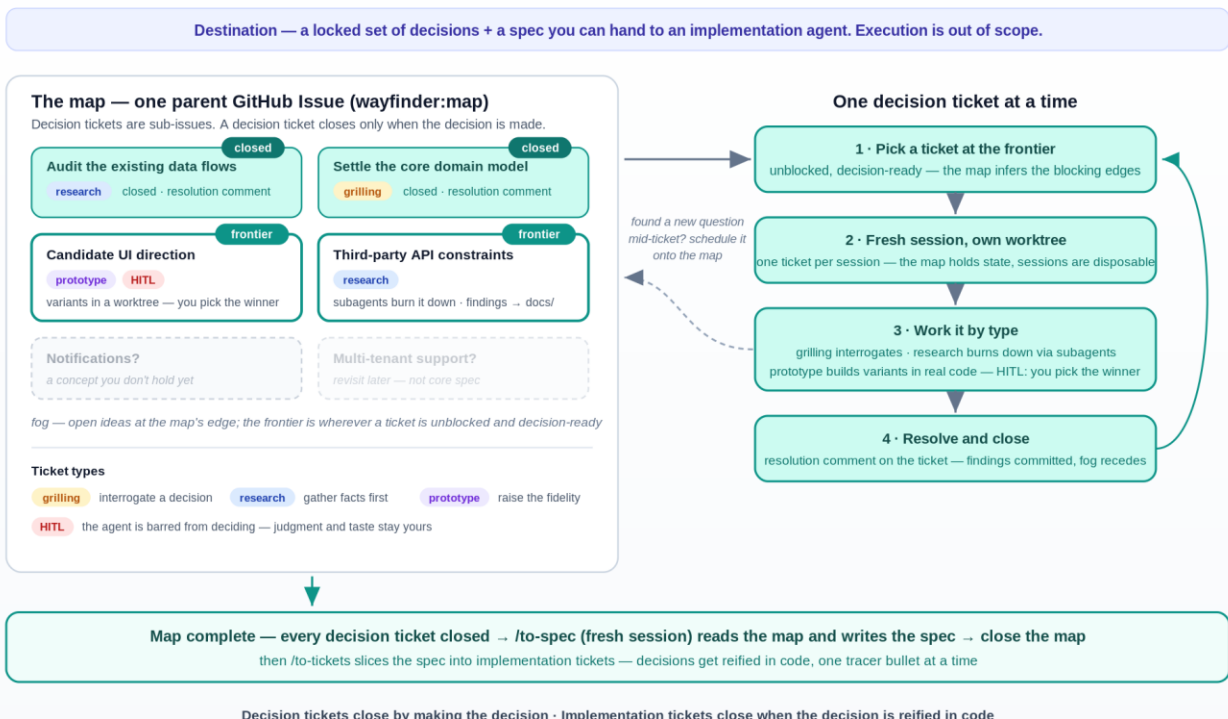
The principle. Spotting that a cluster of modules should become one deep module still requires human architectural insight — the agent can’t do it unaided yet. You become the architect and interface designer; the agent becomes the implementation specialist. And the continuous-improvement loop is the whole point: feed new domain terms, rejected designs, and architectural friction back into the next Grill so the system compounds and gets better.

The Wayfinder Map — when the idea is bigger than a session

Sometimes the loose idea is bigger than any one session can hold — wrapped in fog, with the way from here to the destination not yet visible. Wayfinding is about finding the way, not charging at the destination. /wayfinder charts that way as a shared map on the issue tracker, then works it one ticket per session until the way is clear. It is the second front door of the main flow: where /grill-with-docs sharpens an idea you can hold in one session, wayfinder is for the idea you can't — and in practice it has become Matt's default entry for anything non-trivial, since it degrades gracefully: if it senses no map is needed, it simply runs as a plain grilling session. Where it differs from a plain grill is scale — a greenfield project, a huge feature build — and grilling, research, and prototyping become tickets on a map instead of one-off sessions. The destination is a locked set of decisions plus a spec you can hand to an implementation agent — execution is out of scope: the map produces decisions, not deliverables. When the map is done it hands off, it doesn't build: merge onto the main flow at /to-spec (or straight to /implement if the effort turned out small). Instead of managing session anxiety — handoffs, the zone budget — everything is saved on the tracker: close a session, open the next ticket. The map is done when nothing is left to decide before someone goes and does the thing; the pull to just do the work is usually the signal you have reached the map's edge and it is time to hand off into a spec and Phases 4–6, the closed tickets riding along as primary sources. The handoff is mechanical: point /to-spec at the map in a fresh session, let it read the map and the closed tickets, and write the spec — then close the map; its job is done, and a lingering map is doc rot.

The Wayfinder Map — resolving decisions until the way is clear

Work too big for one session becomes a shared map of decision tickets on GitHub Issues, walked one session at a time.



Wayfinder is the multi-session entry to the workflow — it degrades to a plain grilling session when no map is needed. GitHub-native here: the map and its decision tickets are Issues with wayfinder:<type> labels.

The wayfinder map: typed decision tickets walked one session at a time until the way to the spec is clear.

The map is a single issue on the repo's issue tracker, labelled `wayfinder:map` — the canonical artifact. In this guide's GitHub-native shape, the map is a parent GitHub Issue and every decision ticket is a sub-issue of it, carrying a type label `wayfinder:<type>`. The map is an index, not a store. It names the destination — settled first, because it fixes the scope — keeps a one-line gist plus link for every decision made, and carries two honest lists: a “## Not yet specified” section (the in-scope fog) and a “## Out of scope” section (work consciously ruled beyond the destination, which never graduates back in). Each decision ticket holds one question — a question whose resolution is a decision, not a slice of a build to execute — sized to one agent session, and typed:

- **Research** (`wayfinder:research`, AFK) — resolved by a `/research` subagent working primary sources, each committing a cited findings file on a branch the ticket links to. In Copilot, run these as custom agents or fan them across `/fleet` — or lean on the CLI's experimental native `/research`.
- **Prototype** (`wayfinder:prototype`, HITL) — raise the fidelity of the discussion with a cheap artifact to react to.
- **Grilling** (`wayfinder:grilling`, HITL) — the default type.
- **Task** (`wayfinder:task`, HITL or AFK) — the one type that does rather than decides: manual work that unblocks a decision.

HITL means the ticket only resolves through that live exchange; the agent never stands in for the human's side — a grilling agent that answers its own questions has broken the contract.

The map is deliberately incomplete: don't plan past the fog of war. Beyond the live tickets lies fog — decisions you can tell are coming but can't yet pin down. Fog or ticket? The test is whether you can state the question precisely now — not whether you can answer it now. The frontier is the open, unblocked, unclaimed child tickets — the edge of the known. Resolving one clears the fog ahead of it, graduating newly specifiable questions into fresh tickets; blocking uses the tracker's native dependency relationship — GitHub's issue dependencies between sub-issues — so the frontier renders visually in GitHub's own UI. “Resolve just the decisions at the frontier first.”

The discipline: charting the map is one session's work (grill the destination, grill the frontier breadth-first, create the tickets, wire the blocking edges — and stop); charting fires all `/research` subagents in parallel the moment the map is charted, but hand-resolves nothing. Thereafter never resolve more than one ticket per session — with the exception of research tickets, whose subagents run in parallel — and a session claims a ticket by assigning it to itself FIRST, before working it, so parallel sessions skip it. Each resolution posts the answer as a comment, closes the ticket, and appends a one-line pointer to the map. A ticket can also schedule new tickets mid-flight: when working one decision surfaces a question this session can't hold, write the findings into the current ticket, close it, and open the new question onto the map — the thing a plain grilling session never had anywhere to put. In everything a human reads, refer to tickets by name, never bare ids — a wall of #42, #43, #44 is illegible. If the frontier surfaces no fog at all, the journey fits one session and you don't need a map.

Why it works. The map is progressive disclosure applied to planning: each session loads the low-res map, zooms into exactly one ticket, and leaves one decision behind — so a hundred grilling, prototyping, and research sessions can contribute to one central map that grows as fog clears and shrinks as questions close. (Matt's first real use planned an entire course this way.) And because the map lives on GitHub Issues, the frontier is visible to collaborators in the tracker's own UI — expect other sessions to be editing it concurrently.

Part V · The Skill Library

The library is organized into engineering, productivity, and misc buckets, with three staging buckets around them: in-progress (rough edges and breaking changes expected until a skill graduates), personal (tied to Matt’s own setup), and deprecated (kept for provenance). Within a bucket, skills split by who invokes them, and every public skill has a companion deep-dive doc in the repo’s docs/ folder.

Installing on Copilot CLI: the one-command route is `npmx skills@latest add mattpocock/skills` — the skills.sh installer from Vercel Labs (a third-party tool) — which installs into any harness that speaks the Agent Skills standard, Copilot CLI included. The officially supported surfaces are project directories `.github/skills/`, `.claude/skills/`, or `.agents/skills/`, personal directories `~/.copilot/skills/` or `~/.agents/skills/`, and the in-CLI `/skills add | remove`. Matt’s suite also ships as a native Claude Code plugin (`/plugin marketplace add mattpocock/skills`), but that distribution is Claude-only — on Copilot, use the copy-in installer. Whichever route you take, select and run `/setup-matt-pocock-skills` once per repo: it writes an “### Issue tracker” block into your AGENTS.md so every skill knows where tickets live. Copilot CLI discovers the skills progressively — seeing each skill’s name and description first, and loading its full instructions only when it’s invoked.

How the skills compose

The `/ask-matt` skill is a router over the whole set; it encodes the canonical main flow and the on-ramps to it:

- **Main flow:** `/grill-with-docs` or `/wayfinder` — wayfind the effort that spans sessions, grill the idea that fits one; either way, sharpen it against the codebase — → if multi-session, `/to-spec` → `/to-tickets` → a fresh `/implement` per issue; if it settles in one conversation, just implement in place. Either way `/implement` drives `/tdd` internally and closes out with `/code-review`.
- Keep the grill → spec → tickets chain in ONE unbroken context window — don’t compact or clear until after `/to-tickets`, because that window is full of design gold.
- **On-ramps:** a pile of incoming bugs/requests → `/triage` (only for issues you didn’t create yourself); codebase health → `/improve-codebase-architecture`; a side task mid-flow → `/handoff` (fork to a new session) or compaction (continue the same one); something broken → `/diagnosing-bugs`, whose post-mortem hands off to `/improve-codebase-architecture` when the real finding is a missing seam; reading legwork → `/research`, a background agent whose cited file feeds the next grill; a huge, foggy effort too big for one session → `/wayfinder`, which merges onto the main flow at `/to-spec` (Part IV) — the second front door of the main flow.
- **Vocabulary underneath:** `/domain-modeling` (the project’s domain language) and `/codebase-design` (the deep-module vocabulary) run beneath the other skills — each the single source of truth for its words. Reach for them directly when the words, not the process, are the problem; otherwise the skills above pull them in.
- **Precondition:** `/setup-matt-pocock-skills` once per repo to wire up the issue tracker, triage labels, and domain-doc layout.

Engineering skills

Skill	Invoke	What it does
ask-matt	user	Router/index: asks which skill or flow fits your situation and maps how the skills compose — now with real wayfinder-vs-grilling routing guidance.

grill-with-docs	user	Grilling session that also builds your domain model — sharpens terminology and updates CONTEXT.md and ADRs inline.
to-spec	user	Turns the current conversation into a spec (né PRD: problem, solution, user stories, decisions, out-of-scope) and publishes it. No interview.
to-tickets	user	Breaks a plan, spec, or conversation into tracer-bullet tickets, each declaring its blocking edges — text in a local tickets.md, or native blocking links on GitHub Issues. Merges the old to-plan and to-issues.
triage	user	Moves issues through a state machine of triage roles; preps a clean, agent-ready backlog.
prototype	model	Builds a throwaway prototype — a runnable terminal app for logic, or several UI variations on one route — then captures, doesn't dispose.
improve-codebase-architecture	user	Scans where change is landing for deepening opportunities, presents them as a visual HTML report, then grills the one you pick.
setup-matt-pocock-skills	user	Configures a repo for the engineering skills: issue tracker, triage labels, domain-doc layout. Run once per repo.
tdd	model	Red-green, one vertical slice at a time; tests behavior through interfaces, not implementation.
diagnosing-bugs	model	Disciplined diagnosis loop for hard bugs and perf regressions: build a red-capable loop, reproduce, hypothesize, instrument, fix, regression-test.
domain-modeling	model	Actively builds and sharpens the project's domain model; the reusable DDD discipline behind grill-with-docs.
codebase-design	model	Shared vocabulary and discipline for designing deep modules — the layer tdd, to-spec, and improve-codebase-architecture lean on.
implement	user	Implements a spec or ticket: drives /tdd at the pre-agreed seams, typechecks continuously, runs the full suite once at the end, then /code-review, then commits.
wayfinder	user	Born as decision-map, graduated in v1.1: plans work too big for one session as a shared map of typed decision tickets

		(research / prototype / grilling / task) on the issue tracker — destination, frontier, fog of war — resolved one ticket per session. See Part IV.
code-review	model	Two-axis review of the diff since a fixed point — Standards (repo conventions plus a Fowler smell baseline) and Spec — run as parallel sub-agents and reported side by side, never merged.
research	model	Background agent that investigates a question against primary sources and leaves a cited markdown file in the repo while you keep working. Runs well as a Copilot custom agent; fan several via /fleet.
resolving-merge-conflicts	model	Loop for an in-progress merge/rebase: find each conflict's primary sources, preserve both intents, run the checks, finish — never —abort.

Productivity skills

Skill	Invoke	What it does
grill-me	user	Relentless one-question-at-a-time interview about a plan or design — the general, no-codebase grilling tool.
handoff	user	Compacts the current conversation into a handoff doc (with a suggested-skills section) so another agent can continue.
teach	user	Teaches you a concept over multiple sessions, using the current directory as a stateful teaching workspace.
writing-great-skills	user	Reference for writing and editing skills well — the vocabulary and principles that make a skill predictable.
grilling	model	The reusable interview primitive, reworded in July 2026 for general use on any plan, decision, or idea — decision tree, facts/decisions split, confirmation gate — that grill-me and grill-with-docs are specializations of.

Misc skills

Skill	What it does
git-guardrails-claude-code	Installs hooks that block dangerous git commands (push -force, reset -hard, clean -fd, branch -D) before they run. Claude Code-specific; in Copilot CLI the same effect comes from <code>--deny-tool</code>

	rules, where deny always beats allow.
setup-pre-commit	Husky + lint-staged + Prettier + typecheck + test pre-commit hooks.

Skills to watch

These live in the repo's in-progress bucket — expect rough edges and breaking changes until they graduate, the way wayfinder did in v1.1 (it began life there as decision-map).

Skill	What it does
loop-me	Stateful /grilling workspace that turns recurring “loops” in your work/life into implementable workflow specs — triggers, checkpoints, decision-ready briefs, with the checkpoint pushed as far right as it will go.
wizard	Generates an interactive bash wizard that walks a human through a manual procedure — opening URLs, capturing values, writing .env files and CI secrets — from a battle-tested template; only the stages are authored per run.
claude-handoff	Hands the current conversation to a fresh background agent immediately (<code>claude --bg</code>), seeded with the handoff summary as its prompt. Claude Code-specific.
setup-ts-deep-modules	Sets up a TypeScript package for deep modules: lib/ + tests/ as the canonical subfolders, an entry-points model over a single barrel, and a conventions README the steering files point at.

Part VI · Putting It Into Practice

Phase 0 — setup (once per machine, once per repo)

- **Install the skills:** `npx skills@latest add mattpocock/skills` (the Vercel Labs `skills.sh` installer — third-party, but it targets the Agent Skills standard Copilot reads), or drop the skill folders into `.github/skills/` (project) or `~/.copilot/skills/` (personal), or use `/skills add` in the CLI. The Claude Code plugin distribution is Claude-only; on Copilot, copy-in is the route.
- **Stand up the supporting system:** run `/setup-matt-pocock-skills` once inside each repo. It walks you through three choices — issue tracker (GitHub / GitLab / local markdown / other), the triage label vocabulary, and where domain docs live — and writes an “### Issue tracker” block into your `AGENTS.md`. Do this before first using `to-spec`, `to-tickets`, `triage`, `tdd`, or `improve-codebase-architecture`.
- **Add the deterministic safety net:** run `/setup-pre-commit`, and set standing `--deny-tool` rules (deny always beats allow) so formatting, type errors, and dangerous git commands never reach a commit.

Greenfield vs brownfield

On an existing (brownfield) codebase, start with `/grill-with-docs` to build the glossary as you plan your first change. On a new (greenfield) project, build the supporting docs at the earliest possible moment — grill, save findings to research files, prototype the risky ideas, define the module shape, then write the spec — so everything downstream already speaks the right domain language. Establishing `CONTEXT.md` and ADRs during the initial grill (not after the spec) is what keeps later tickets from using stale terminology. Alternatively, use `/wayfinder` to plot the course for your whole app. Don’t over-index on the distinction, though — “brownfield versus greenfield is kind of BS”: the workflow is identical, the fundamentals transfer, and `wayfinder` runs as happily on a decade-old codebase as on an empty folder. What changes is only where the first domain terms come from.

From HITL to AFK — earn the autonomy

Begin human-in-the-loop: watch the agent work a slice, build trust, and feel where your taste is required (planning, QA, architecture, UX). As trust grows, move well-defined AFK slices into autopilot or the Ralph loop in a sandbox, and parallelize — plan the next feature or QA another run while the agent ships. HITL and AFK are first-class citizens of the same workflow; orchestrate a hybrid rather than choosing one.

The grill / execute / clear rhythm

For a small feature or bug fix that fits one window: grill to alignment, implement, QA in the same window, then clear and move on. For anything larger: grill → spec → tickets in one unbroken window, then run each issue in a fresh context. Clear early — around 34–40% of the window in HITL implementation — accepting the small cost of a re-explore in exchange for staying firmly in the efficient zone.

Working as a team

The workflow is collaborative up to the spec and single-owner after it. Grill as a group — mob grilling with a domain expert in the room (or a meeting transcript pasted into the session) is the highest-bandwidth way to load the design concept — then let one engineer own the implementation, because a shared HITL session has no pilot. Standards are a team asset: keep one company-wide set, mined

from real PR review comments, living in the reviewer. Team knowledge that outlives any one repo — the shared glossary, ADRs, research findings — belongs in its own wiki repo (or the GitHub wiki, or a third-party service like Notion or Google Docs), not in local markdown scratch files that go stale invisibly. On GitHub the wayfinder map compounds this: because the map and its sub-issues live on the shared tracker, the frontier is visible to every collaborator, and claiming-by-assignment is what keeps parallel sessions from colliding.

Maintenance cadence

- **Every couple of days in a fast-moving repo (weekly at minimum):** run `/improve-codebase-architecture` after a surge of development to find deepening opportunities before entropy compounds.
- **Every couple of weeks:** audit your auto-generated memory, `research.md`, ADRs, and the glossary for staleness — prune anything that no longer matches the repo and disable memory for coding repos.
- **Every run:** keep the AFK agent to one task per iteration, with a ready-for-agent label gating what it may pick up.
- **Continuously:** feed new domain terms, rejected designs, and architectural friction back into the next Grill — and mine `/chronicle` for workflow friction. That loop is the entire point.

The through-line. You are the architect, interface designer, and keeper of taste; the agent is the implementation specialist working inside the rails you lay. Get the constraints right, encode your judgment as skills and domain docs, and “AI coding gives you permission to build bigger” — without building a ball of mud you can no longer change.

Part VII · Sandcastle — The AFK Software Factory

Part IV ended Phase 6 with a promise: once you trust the loop, you go AFK — away from keyboard — and let agents pull work from a queue. Sandcastle is the machine that makes that safe and repeatable. It is Matt’s open-source answer to the question “how do I run coding agents unattended without handing them the keys to my laptop?” This chapter is a standalone guide to it.

Why Sandcastle exists

Running an agent unattended has two failure modes. The first is danger: an agent in “yolo” mode (all permissions granted, no human in the loop) can delete your home directory, leak secrets, or run destructive commands. The second is friction: the existing ways to sandbox an agent — raw Docker, vendor cloud tools — took a lot of configuration and were hard to own. Matt wanted something that was simple, scriptable, agent-agnostic, and entirely yours: just code you can read and change. That is Sandcastle.

The one-line description from the project: a TypeScript library for orchestrating AI coding agents in isolated sandboxes. You invoke an agent with a single call; Sandcastle handles isolating it with a configurable branch strategy; the commits it makes get merged back. It is good for parallelizing multiple AFK agents, building review pipelines, or just orchestrating your own agents.

The mental model and vocabulary

Sandcastle ships a deliberate, precise vocabulary (it practices the ubiquitous-language discipline the rest of this guide preaches). The terms worth knowing before you read any example:

- **Sandcastle** — the tool that orchestrates an agent inside a sandbox.
- **Agent** — the AI coding tool invoked inside the sandbox (GitHub Copilot CLI, Claude Code, Codex, and others). It is swappable; nothing below is harness-specific.
- **Sandbox** — the isolation boundary around the agent: a container, VM, or similar environment that constrains what it can touch.
- **Host** — your machine, where Sandcastle runs and the real git repo lives.
- **Sandbox provider** — a pluggable implementation that creates the sandbox, passed to `run()` via the `sandbox` option. Three shapes: a `bind-mount` provider mounts the host filesystem into the environment (Docker, Podman); an `isolated` provider has its own filesystem and syncs code in and commits out (Vercel); a `no-sandbox` provider runs the agent directly on the host with no container.
- **Branch strategy** — how the agent’s changes relate to git branches. `head` works directly in the host working directory; `merge-to-head` creates a temporary branch, lets the agent work on it, then merges back to your current branch; `branch` lands commits on an explicitly named branch you provide.
- **Worktree** — a git worktree under `.sandcastle/worktrees/` on the host, used by the `merge-to-head` and `branch` strategies so the agent never works directly on your checked-out branch.
- **Iteration** — a single invocation of the agent inside the sandbox, producing at most one commit against one task (a work item the agent selects, typically from an issue tracker).
- **Completion signal** — a marker the agent emits (default `<promise>COMPLETE</promise>`) to end the iteration loop early. Structured output is the orthogonal idea: a schema-validated JSON payload the agent emits inside a named XML tag, returned to the caller.

Quick start

Sandcastle is an npm package. Install it, scaffold a config directory, add your token, and run a TypeScript file:

```
npm install --save-dev @ai-hero/sandcastle
npx @ai-hero/sandcastle init      # scaffolds a .sandcastle/ directory
# edit .sandcastle/.env: set a GitHub token (COPILOT_GITHUB_TOKEN / GH_TOKEN)
# (a fine-grained PAT with the Copilot Requests permission, or any token from `gh auth login`)
npx tsx .sandcastle/main.ts      # run the agent
```

The init command interactively asks three questions: which agent (GitHub Copilot CLI, Claude Code, Codex), which sandbox provider (Docker is the first-class local option), and which issue tracker / backlog (GitHub issues is the default; issues are filtered by a sandcastle label). It writes a .sandcastle/ folder containing a Dockerfile, an .env, and a main.ts you run with tsx.

The smallest meaningful program is one run() call:

```
import { run, copilot } from "@ai-hero/sandcastle";
import { docker } from "@ai-hero/sandcastle/sandboxes/docker";
await run({
  agent: copilot("claude-opus-4-8"),
  sandbox: docker(),           // or podman(), vercel(), noSandbox()
  promptFile: ".sandcastle/prompt.md",
});
```

The run() API, in brief

run() takes an agent and a sandbox (both required) plus a prompt (inline prompt or promptFile). The options you reach for most:

- **maxIterations** — how many times the agent runs before stopping (default 1). Each iteration works one task and makes at most one commit.
- **branchStrategy** — head, merge-to-head, or an explicit branch (defaults: head for bind-mount providers, merge-to-head for isolated ones).
- **hooks** — lifecycle commands grouped by where they run: host hooks (e.g. copy an .env into the worktree) and sandbox hooks (e.g. npm install on onSandboxReady).
- **completionSignal** — the string(s) the agent emits to end the loop early. idleTimeoutSeconds fails a run that has gone silent; completionTimeoutSeconds is the grace window after the completion signal is seen but the process has not yet exited (a “hanging process” — usually a spawned gh/git child holding stdout open).
- **output** — Output.object({ tag, schema }) or Output.string({ tag }) to extract typed, schema-validated structured output from the agent’s stdout (requires maxIterations === 1, and the tag must appear in the prompt).

run() returns the iterations executed (with per-iteration usage), the commits created, and the target branch — so a harness can inspect exactly what happened.

Sandbox providers

The sandbox option accepts any provider; the built-ins cover most needs:

- **Docker** — bind-mount; the most common local choice, shown in nearly every example.
- **Podman** — bind-mount; a rootless alternative to Docker.
- **Vercel** — isolated; cloud Firecracker microVMs via @vercel/sandbox.

- **no-sandbox** — no container at all; the agent runs on the host. Useful when container isolation is undesirable, but you lose the safety the rest of this chapter is about.

You can also build your own provider with `createBindMountSandboxProvider` or `createIsolatedSandboxProvider`.

The pipeline it unlocks: plan → implement → review → merge

A single `run()` is the primitive. The pattern demonstrates is a multi-agent factory built from it, mirroring the seven-phase workflow:

- **Planner.** One agent reads the open, labeled issues and emits a plan (wrapped in a tag, as structured output) of the unblocked issues to work — a DAG, not a flat list.
- **Implementers, in parallel.** Each issue spawns its own agent on its own branch, prompted with the issue title and id, working red-green via `/tdd`.
- **Reviewer.** A reviewer agent runs against each branch's diff, checking it against the spec and the repo's coding standards — the two-axis `/code-review` discipline — fixing problems, and reporting to the human. (Use a different, sometimes more expensive, model here than for implementation.)
- **Merger.** A final agent pulls the branches back to the main branch, resolving merge conflicts via `/resolving-merge-conflicts` and closing issues with comments.

Because it is “just code,” you can rearrange these freely: adversarial cross-agent review (implement with one model, review with another), or several agents proposing competing implementations while a reviewer picks the best.

For the common implement-then-review sequence, `createSandbox()` keeps a single container warm across multiple `run()` calls, so dependencies and build artifacts persist instead of paying container boot each phase. `sandbox.exec()` runs a shell command in that same warm sandbox — handy to gate the review on a quick test pass:

```
await using sandbox = await createSandbox({ branch: "agent/fix-42", sandbox: docker() });
await sandbox.run({ agent: copilot("claude-opus-4-8"), promptFile: ".sandcastle/implement.md",
maxIterations: 5 });
const tests = await sandbox.exec("npm test"); // non-zero exitCode is returned, not thrown
if (tests.exitCode !== 0) throw new Error(tests.stdout);
await sandbox.run({ agent: copilot("gpt-5.5"), prompt: "Review the changes and fix any issues." });
```

How it fits the workflow

Sandcastle is the engine behind Phase 6 (Implement: HITL first, then AFK) and Phase 7 (Review). The connective tissue is triage: an issue labeled ready-for-agent, with an agent brief attached, is exactly what the planner picks up. This is a queue, not an infinite “Ralph loop” — bounded iterations and a real backlog, which keeps token spend predictable and keeps a human checkpoint where it belongs (at review, pushed as far right as you dare).

Native autopilot and the Copilot cloud agent

Sandcastle is the heavyweight option; Copilot CLI can also run AFK on its own. Inside a dev container with a ready-for-agent backlog in GitHub Issues, a single headless command pulls and ships one issue:

```
copilot -p "Pull the top ready-for-agent issue, implement it red-green, run
the tests, commit with a detailed message, and close it. One issue, then stop." \
--autopilot --allow-all-tools --max-autopilot-continues 40 \
--deny-tool='shell(git push)' --deny-tool='shell(git reset --hard)' \
-s --share
```

`--allow-all-tools` grants tool access, which headless runs must do explicitly (or set `COPILOT_ALLOW_ALL`) — safe only because of the container; `--deny-tool` rules are deterministic guardrails that always beat allow; `--max-autopilot-continues` caps the loop, which is otherwise unlimited; `-s` keeps output scriptable and `--share` leaves a session log a human can review. Use `/fleet` to fan several issues across parallel subagents.

Push the human checkpoint further right by delegating to the Copilot cloud agent — renamed from “Copilot coding agent” in April 2026: assign a labelled issue to Copilot and it runs on an ephemeral GitHub Actions runner, opens a draft pull request, and responds to review comments — no longer limited to PR workflows, and since July 2026 needing no PAT inside Actions — the same plan → implement → review → merge shape Sandcastle gives you locally, but hosted and tied to your existing branch protection and CI. Sandcastle, native autopilot, and the cloud agent are three points on one spectrum: reach for whichever matches how much you want to own versus hand to GitHub.

Sidebar: running Sandcastle with GitHub Copilot CLI

Copilot CLI is a first-class, shipping agent provider in Sandcastle — `copilot()` sits beside `claudeCode()` and `codex()`, and for each iteration it runs `copilot -p` with `--allow-all-tools` and your chosen `--model` inside the sandbox. A few Copilot-specific details are worth knowing:

- **Install Copilot CLI inside the image, not just on the host.** The init command already does this — its Dockerfile runs `RUN npm install -g @github/copilot` — but if you hand-roll the image, add that line or you get `sh: copilot: command not found`.
- **Pass a GitHub token into the container.** Drop a token with the Copilot Requests permission into `.sandcastle/.env`; Sandcastle reads it in precedence `COPILOT_GITHUB_TOKEN > GH_TOKEN > GITHUB_TOKEN`, and the bundled `gh` uses the same token to read your GitHub Issues backlog.
- **Expect no session resume.** Copilot CLI indexes session state in a SQLite DB, so Sandcastle marks the `copilot()` provider non-resumable: multi-iteration runs still work, but each iteration starts fresh with no conversation carryover (often what you want anyway) and there is no captured per-iteration token usage.
- **Keep prompts under ~120 KB.** The prompt is passed via the `-p argv` argument, not `stdin`, so very large prompts throw a clear guard error — keep them lean and push detail into `research.md` and the issue bodies the agent reads itself. (And don’t copy host `node_modules` into a Linux container; let it run `npm install`.)

Gotchas worth knowing

- **Worktree locking.** `git` refuses to check out the same branch in two worktrees at once; if a branch is already checked out in your main working tree, pick a different branch or switch first.
- **Hanging processes vs stuck agents.** A finished agent whose child process keeps `stdout` open is resolved by `completionTimeoutSeconds` (a success, with a warning), distinct from a genuinely silent agent that hits the idle timeout (a failure).
- **Resume and structured-output retries (v0.11).** `sandbox.run` supports resuming a session inside a warm container, and `Output.object/Output.string` gained `maxRetries` that re-feeds extraction errors to the agent — both require a resume-capable agent provider (`claudeCode`, `codex`); the `copilot()` provider is non-resumable, so it skips these.

When to use it — and when not

Reach for Sandcastle when you have a real backlog of well-specified, independent issues and you trust your feedback loops (types, tests, review) enough to let agents run unattended. Stay human-in-the-loop — and skip the AFK machinery — while you are still grilling, prototyping, or working on anything whose

spec is not yet hard. Sandcastle does not replace the discipline in Parts I–IV; it is what you graduate to once that discipline is in place.

Appendix A - Vocabulary

Term	Meaning
Efficient zone / inefficient zone	Early in a session the agent is sharp; as context fills it drifts into sloppier, forgetful output. The efficient zone is roughly the first 100k tokens; the inefficient zone commonly sets in around 125–150k (“smart/dumb zone” in Matt’s dictionary, credited to Dex Horthy). Matt’s own working ceiling: keep a day-task session under ~150k.
Attention budget	Each token’s finite influence over the rest of the context. Per-token, and it does not grow as context grows.
Harness	Everything around the model that makes it an agent: tools, system prompt, context management, permissions, hooks.
The rings	Model $\subset$ harness $\subset$ agent $\subset$ environment. Model plus harness is the agent; the environment wraps it. Leverage splits ~50/50 between the model and everything around it — you own the environment and choose the harness.
Session / turn	A session is one bounded run from empty to cleared/closed; a turn is one user message plus all the agent does before yielding back.
Subagent	An agent spawned by another via a tool call; its own session, reports a single summary. Subagents can nest several levels deep, enabling model tiering (big-model orchestrator → mid-size workers → small explorers). In Copilot CLI: custom agents, /fleet, /subagents.
Clearing / compaction / handoff	Three ways to deal with a full window: start fresh (preferred); summarize in place (lossy, leaves sediment); or write to a file for a fresh agent. Handoff forks; compaction continues.
Progressive disclosure	Load only the context needed now, with pointers to the rest. The basis of skills — and of the wayfinder map.
Ubiquitous language	A shared domain vocabulary used by people and code alike, captured in CONTEXT.md (Eric Evans, DDD).
ADR	Architectural Decision Record — a short markdown note of a hard, surprising, irreversible decision and the alternatives considered.
Decision tree	Brooks’s design tree, renamed for the grill: the branching set of decisions a plan must walk before code. “Design tree” in earlier editions.

Tracer bullet	A small end-to-end vertical slice through all layers, built and tested before expanding (Pragmatic Programmer).
Deep module	A thin interface over a large hidden implementation (Ousterhout). The core of an AI-friendly codebase.
Seam	A place where behavior can change without editing in place (Michael Feathers).
Grey box	A deep module you can look inside but trust by default — you review its tests more than its internals.
HITL / AFK	Human-in-the-loop (you pair with the agent) vs away-from-keyboard (you kick it off and leave). In a wayfinder ticket, HITL means it only resolves through that live exchange — the agent never stands in for the human's side.
Ralph loop	Geoffrey Huntley's pattern: a bash loop running the same prompt repeatedly, each iteration in a fresh context window.
AX	Agent experience — how well the environment (checks, architecture, free context) lets an agent do good work.
Automated check / review / human review / vibe coding	Deterministic pass-fail; an agent judging an agent; a human reading the diff; and accepting code with no review at all.
Wayfinder map / destination	What a wayfinder effort is finding its way to (a locked set of decisions plus a spec — execution out of scope), and the tracker issue that indexes the route: decisions so far, live tickets, fog, out-of-scope. On GitHub: a parent issue labelled wayfinder:map with sub-issue decision tickets.
Frontier / fog of war	The open, unblocked, unclaimed tickets at the edge of the known; and the in-scope decisions you can't yet phrase sharply (the map's "## Not yet specified" section). "Don't plan past the fog of war."
Decision ticket / implementation ticket	A wayfinder ticket closes only by making its decision; an implementation ticket (from /to-tickets) closes when the decision is reified in code. Decision tickets chart the way; implementation tickets walk it.
Capture, don't dispose	Prototype doctrine: fold the validated decision into real code, commit the prototype to a throwaway branch off main as a primary source, and leave a context pointer on the implementation issue — main keeps only the decision.

Expand–contract	How to slice a wide refactor: add the new form beside the old, migrate call sites in green batches, delete the old form last.
Leading word	A short, loaded phrase (“tracer bullet,” “vertical slice”) that steers the agent with a concept it already knows; hearing it repeated back is your verification.

Appendix B - The Reading List

The workflow is, in effect, an applied reading list. These are the texts it draws on:

- **The Pragmatic Programmer** — David Thomas & Andrew Hunt. Tracer bullets, “no-one knows exactly what they want,” small deliberate steps, software entropy.
- **The Design of Design** — Frederick P. Brooks. The shared design concept and the design tree — grilling’s decision tree — behind it.
- **Domain-Driven Design** — Eric Evans. Ubiquitous language, bounded contexts, context maps.
- **A Philosophy of Software Design** — John Ousterhout. Deep modules and “design it twice.”
- **Extreme Programming Explained** — Kent Beck. “Invest in the design of the system every day” and the TDD lineage.
- **Working Effectively with Legacy Code** — Michael Feathers. Seams.
- **Refactoring** — Martin Fowler. Keeping changes small and the suite green; chapter 3’s code smells are /code-review’s Standards baseline.
- **Shape Up** — Ryan Singer. The fidelity ladder (text → wireframe → prototype → real app) behind prototyping at the right resolution.

Appendix C - Command Quick Reference

Command	Phase / use
/setup-matt-pocock-skills	0 — configure tracker, labels, domain docs (once per repo); writes the “### Issue tracker” block into AGENTS.md
/setup-pre-commit	0/6 — deterministic typecheck + test + format hooks
/skills add remove	0 — manage installed skills in-CLI (project dirs: .github/skills, .claude/skills, .agents/skills; personal: ~/.copilot/skills, ~/.agents/skills)
/grill-with-docs / /grill-me	1 — harden the idea; with-docs also builds CONTEXT.md + ADRs
/wayfinder	1–3 — chart an idea too big for one session as a map of typed decision tickets on GitHub Issues; one ticket per session (research runs parallel via subagents)
/research	2 — background agent; primary sources → a cited markdown file (Copilot also has an experimental native /research)
/prototype	3 — throwaway UI variations or a logic terminal app; capture, don't dispose
/to-spec	4 — synthesize the conversation into a spec (né PRD)
/to-tickets / /triage	5 — slice into tracer-bullet tickets with blocking edges; groom the backlog
/tdd / /diagnosing-bugs	6 — red-green at the agreed seams; disciplined bug diagnosis
/implement	6 — build a spec/ticket: /tdd inside, /code-review to close, then commit
/handoff	6 — compact a session to a file for a fresh agent
/resolving-merge-conflicts	6 — resolve an in-progress merge/rebase from primary sources; never –abort
/code-review	7 — two-axis review (Standards + Spec) of the diff since a fixed point; owns the refactor

<code>/improve-codebase-architecture</code>	7 — find deepening opportunities; visual HTML report
<code>/ask-matt</code>	any — router: which skill or flow fits your situation
<code>autopilot · --autopilot · --mode=interactive plan autopilot · --plan</code>	6 — native AFK loop; Shift+Tab cycles standard → plan → autopilot
<code>-p · --max-autopilot-continues N · --no-ask-user · -s/--silent · --share/--share-gist</code>	6 — headless runs: prompt, iteration cap (default unlimited — always cap), no pausing, script output, reviewable session log
<code>/rubber-duck</code>	7 — cross-model second opinion on a plan, code, or tests; the escalation layer for high-stakes changes
<code>/fleet · /subagents</code>	2/6 — fan a goal across parallel subagents; configure per-agent subagent model routing
<code>/agent · --agent=AGENT</code>	any — run a custom agent (.github/agents/NAME.agent.md, ~/.copilot/agents, org-level via .github repo)
<code>/chronicle standup tips improve search reindex</code>	7 — mine session history for workflow improvements
<code>/yolo · --allow-all (= --allow-all-tools --allow-all-paths --allow-all-urls) · --deny-tool='Kind(argument)'</code>	0/6 — permissions and deterministic guardrails (deny beats allow); COPILOT_ALLOW_ALL for programmatic runs
<code>--model=MODEL · COPILOT_MODEL</code>	any — model selection across Claude / GPT / Gemini families; auto = auto-selection
<code>/memory · /after · /every · /review · hooks · plugins</code>	experimental — steering layers, prompt scheduling, native review, harness extension

Sources

This guide synthesizes the following primary sources. Where public materials and the cohort transcripts differed, the most recent public release was treated as authoritative. GitHub Copilot facts were re-validated against GitHub’s documentation on July 14, 2026.

- Matt Pocock, “AI Coding for Real Engineers” cohort lesson transcripts (Days 1–6), aihero.dev.
- Matt Pocock’s X posts (@mattpocockuk) introducing decision maps and /wayfinder, and the “Skills v1.1” release article on aihero.dev (July 2026).
- Matt Pocock, “LIVE: The /wayfinder Demo” (YouTube, streamed July 13, 2026) — source of the rings model, the decision-vs-implementation-ticket distinction, and the wayfinder session discipline in Parts II and IV.
- [mattpocock/skills](https://github.com/mattpocock/skills) repository, v1.0.1 through main @ 2026-07-13 (commit 66898f6, post-v1.1.0, unreleased) — README, CHANGELOG, docs/, and the individual SKILL.md files.
- Matt Pocock’s aihero.dev writing and YouTube (@mattpocockuk), including “5 Agent Skills I Use Every Day,” “How To Make Codebases AI Agents Love,” “9 Ways AI Coding Has Rewired My Brain,” “Tracer Bullets,” the grill-with-docs and triage skill posts, and the AI Coding Dictionary.
- Engineering texts cited throughout (Appendix B).
- Burke Holland, “10 AI Coding Tips That Actually Work” (YouTube) — the GitHub Copilot CLI workflow woven into this edition.
- GitHub Copilot documentation (docs.github.com and the GitHub changelog) — CLI GA, autopilot and headless flags, permissions, skills and custom-agent directories, chronicle, the Copilot cloud agent (renamed April 2026), VS Code browser tools (GA July 1, 2026), and the experimental /research, /review, /memory, scheduling, hooks, and plugins features.
- [mattpocock/sandcastle](https://github.com/mattpocock/sandcastle) — the copilot() agent provider, init/auth, and run() API (github.com/mattpocock/sandcastle).